

Отчёт по лабораторной работе №5

Исследование задачи обедающих философов средствами сетей Петри

true

Цель работы

Реализовать на языке Julia модель сетей Петри для задачи обедающих философов в классическом и арбитражном вариантах. Научиться обнаруживать тупиковые ситуации (deadlock), визуализировать динамику маркировки сети и проводить параметрическое исследование при разном числе философов.

Задание

- Создать рабочий каталог лабораторной работы и инициализировать проект DrWatson.
- Реализовать модуль DiningPhilosophers со структурой PetriNet и функциями построения классической сети и сети с арбитром.
- Реализовать функции стохастического моделирования simulate_stochastic, проверки тупика detect_deadlock и построения графиков plot_marking_evolution.
- Запустить базовый эксперимент для классической сети и сети с арбитром при $N = 5$, сравнить наличие тупика.
- Построить анимацию изменения маркировки сети для $N = 3$.
- Сформировать итоговый отчёт со сравнительными графиками для обоих вариантов сети.
- Выполнить параметрическое исследование при $N = 3, 5, 7$.
- Преобразовать скрипты в литературный стиль с помощью Literate.jl и сгенерировать Jupyter Notebook и Quarto-документы.

Теоретическое введение

Задача обедающих философов

Задача обедающих философов — классическая иллюстрация проблем синхронизации параллельных процессов: конкуренции за общие ресурсы (вилки) и возможности возникновения тупиковой ситуации (deadlock), когда каждый философ удерживает один ресурс и ждёт второй.

Сети Петри

Сеть Петри задаётся набором позиций (places), переходов (transitions) и матрицей инцидентности, описывающей связи между ними. Маркировка сети — распределение фишек по позициям — представляет текущее состояние системы. Срабатывание перехода переносит фишки из входных позиций в выходные согласно матрице инцидентности.

Для каждого философа i вводятся позиции состояний: Think_i , Hungry_i , Eat_i , а также позиция вилки Fork_i . Переходы соответствуют действиям «захотел есть», «взял вилки», «закончил есть».

В классической сети каждый философ может одновременно претендовать на обе соседние вилки, что при определённой последовательности срабатываний переходов приводит к тупику — все философы одновременно удерживают по одной вилке и бесконечно ждут вторую. В сети с арбитром добавляется дополнительный механизм, ограничивающий число одновременно «голодных» философов, что устраняет возможность тупика.

Таблица пакетов Julia, используемых в лабораторной работе:

Table 1: Пакеты Julia, используемые в лабораторной работе {#tbl-packages}

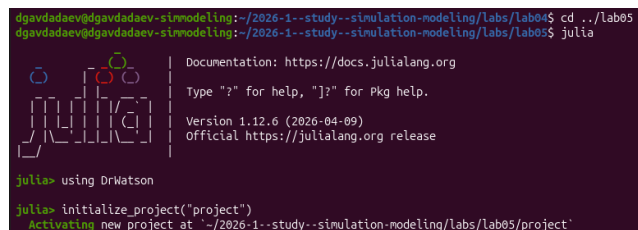
Пакет	Назначение
DrWatson	Организация проекта и воспроизводимость экспериментов
OrdinaryDiffEq	Решение дифференциальных уравнений (зависимость проекта)
Plots	Визуализация графиков и анимаций

Пакет	Назначение
DataFrames	Работа с табличными данными
CSV	Сохранение и загрузка результатов моделирования

Выполнение лабораторной работы

Инициализация проекта

Перешёл в каталог `labs/lab05` и запустил Julia 1.12.6. Выполнил `using DrWatson` и `initialize_project("project")` — создана структура проекта DrWatson для лабораторной работы 5 (рис. @fig-001).



```

dgavdadaev@dgavdadaev-simmodeling:~/2026-1--study--simulation-modeling/labs/lab05$ cd ../lab05
dgavdadaev@dgavdadaev-simmodeling:~/2026-1--study--simulation-modeling/labs/lab05$ julia

Documentation: https://docs.julialang.org
Type "?" for help, "]" for Pkg help.
Version 1.12.6 (2026-04-09)
Official https://julialang.org release

julia> using DrWatson
julia> initialize_project("project")
Activating new project at "~/2026-1--study--simulation-modeling/labs/lab05/project"

```

Figure 1: Инициализация проекта DrWatson для lab05

Модуль DiningPhilosophers

В файле `src/DiningPhilosophers.jl` реализован модуль с базовой структурой PetriNet, содержащей число позиций `n_places`, число переходов `n_transitions`, матрицу инцидентности `incidence` и имена позиций/переходов. Модуль экспортирует функции `build_classical_network`, `build_arbiter_network`, `simulate_ode`, `simulate_stochastic`, `detect_deadlock` и `plot_marking_evolution` (рис. @fig-002).

Базовый скрипт моделирования

В скрипте `scripts/dining_philosophers.jl` для $N = 5$ философов и $t_{\max} = 50.0$ строятся классическая сеть (`build_classical_network`) и сеть с арбитром (`build_arbiter_network`). Для каждой сети выполняется стохастическое моделирование `simulate_stochastic`, результаты сохраняются в CSV (`dining_classic.csv`, `dining_arbiter.csv`), проверяется наличие тупика функцией `detect_deadlock`, строится график маркировки `plot_marking_evolution`

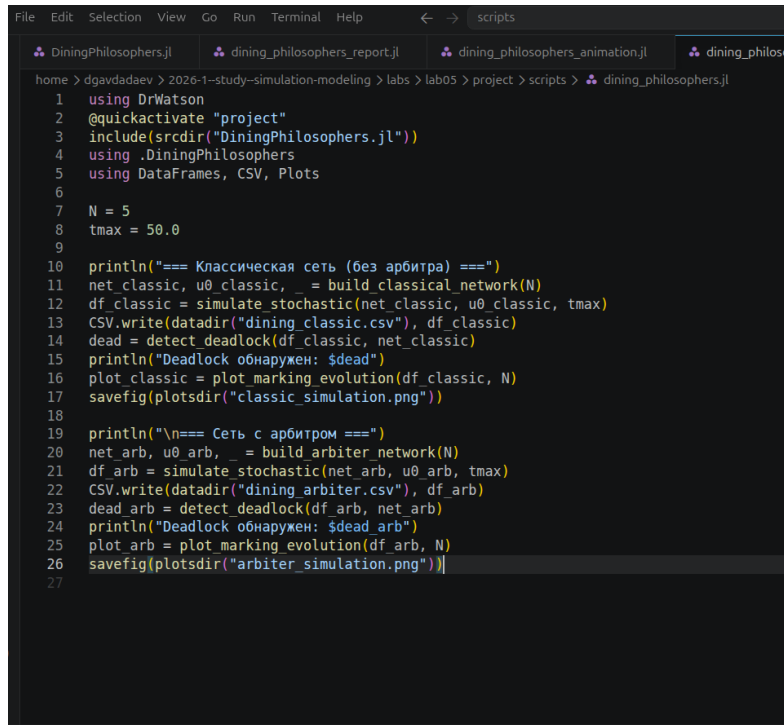
```

File Edit Selection View Go Run Terminal Help ← → scripts
DiningPhilosophers.jl X dining_philosophers_report.jl dining_philosophers_animation.jl dining_philosophers_report.jl
home > dgavdadaev > 2026-1--study--simulation-modeling > labs > lab05 > project > src > DiningPhilosophers.jl
1  module DiningPhilosophers
2
3  using OrdinaryDiffEq
4  using Plots
5  using DataFrames
6  using Random, LinearAlgebra
7
8  export build_classical_network, build_arbiter_network
9  export simulate_ode, simulate_stochastic
10 export detect_deadlock, plot_marking_evolution
11
12 # Определение простой структуры PetriNet
13 struct PetriNet
14     n_places::Int
15     n_transitions::Int
16     incidence::Matrix{Int}
17     place_names::Vector{Symbol}
18     transition_names::Vector{Symbol}
19 end
20
21 function PetriNet(
22     n_places,
23     n_transitions;
24     place_names = Symbol[],
25     transition_names = Symbol[],
26 )
27     incidence = zeros{Int, n_places, n_transitions}
28     if isempty(place_names)
29         place_names = [Symbol("p$i") for i = 1:n_places]
30     end
31     if isempty(transition_names)
32         transition_names = [Symbol("t$i") for i = 1:n_transitions]
33     end
34     PetriNet(n_places, n_transitions, incidence, place_names, transition_names)

```

Figure 2: Структура PetriNet в модуле DiningPhilosophers.jl

и сохраняется в `classic_simulation.png` и `arbiter_simulation.png` ([рис. @fig-003]).



```
File Edit Selection View Go Run Terminal Help ← → scripts
DiningPhilosophers.jl dining_philosophers_report.jl dining_philosophers_animation.jl dining_philosophers.jl
home > dgavdadaev > 2026-1-study-simulation-modeling > labs > lab05 > project > scripts > dining_philosophers.jl
1 using DrWatson
2 @quickactivate "project"
3 include(sourcedir("DiningPhilosophers.jl"))
4 using .DiningPhilosophers
5 using DataFrames, CSV, Plots
6
7 N = 5
8 tmax = 50.0
9
10 println("=== Классическая сеть (без арбитра) ===")
11 net_classic, u0_classic, _ = build_classical_network(N)
12 df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
13 CSV.write(datadir("dining_classic.csv"), df_classic)
14 dead = detect_deadlock(df_classic, net_classic)
15 println("Deadlock обнаружен: $dead")
16 plot_classic = plot_marking_evolution(df_classic, N)
17 savefig(plotsdir("classic_simulation.png"))
18
19 println("\n=== Сеть с арбитром ===")
20 net_arb, u0_arb, _ = build_arbiter_network(N)
21 df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
22 CSV.write(datadir("dining_arbiter.csv"), df_arb)
23 dead_arb = detect_deadlock(df_arb, net_arb)
24 println("Deadlock обнаружен: $dead_arb")
25 plot_arb = plot_marking_evolution(df_arb, N)
26 savefig(plotsdir("arbiter_simulation.png"))
27
```

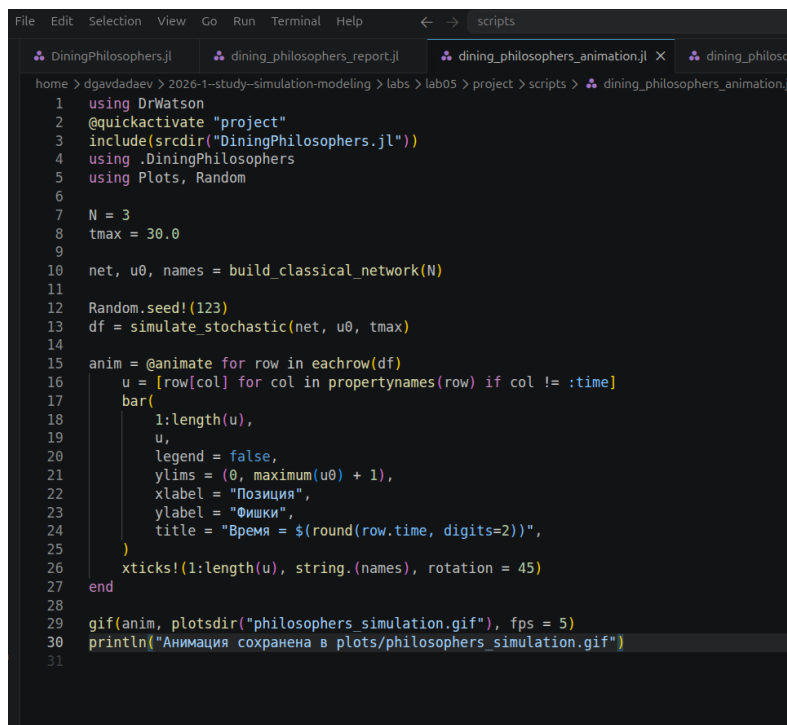
Figure 3: Скрипт `dining_philosophers.jl`: базовый эксперимент для $N = 5$

Скрипт анимации

В скрипте `dining_philosophers_animation.jl` для $N = 3$ философов и $tmax = 30.0$ строится классическая сеть, фиксируется `Random.seed!(123)`, выполняется `simulate_stochastic`. Для каждой строки результирующего датафрейма строится столбчатая диаграмма значений всех позиций сети, кадры собираются в анимацию с помощью макроса `@animate` и сохраняются в `philosophers_simulation.gif` с частотой 5 кадров в секунду ([рис. @fig-004]).

Скрипт формирования итогового отчёта

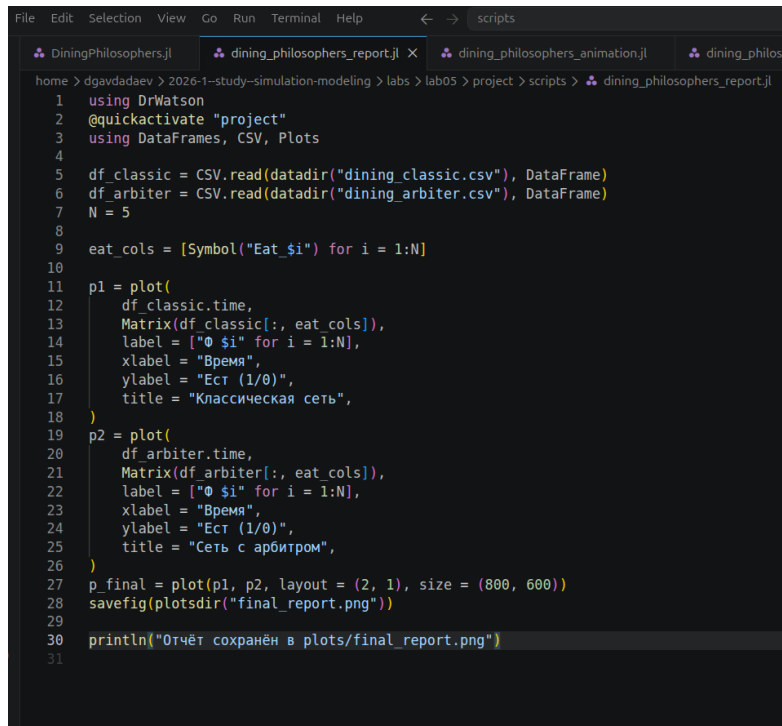
В скрипте `dining_philosophers_report.jl` загружаются результаты `dining_classic.csv` и `dining_arbiter.csv` для $N = 5$. Для каждой



```
1 using DrWatson
2 @quickactivate "project"
3 include(sourcedir("DiningPhilosophers.jl"))
4 using .DiningPhilosophers
5 using Plots, Random
6
7 N = 3
8 tmax = 30.0
9
10 net, u0, names = build_classical_network(N)
11
12 Random.seed!(123)
13 df = simulate_stochastic(net, u0, tmax)
14
15 anim = @animate for row in eachrow(df)
16     u = [row[col] for col in propertynames(row) if col != :time]
17     bar(
18         1:length(u),
19         u,
20         legend = false,
21         ylims = (0, maximum(u0) + 1),
22         xlabel = "Позиция",
23         ylabel = "Фишки",
24         title = "Время = $(round(row.time, digits=2))",
25     )
26     xticks!(1:length(u), string.(names), rotation = 45)
27 end
28
29 gif(anim, plotsdir("philosophers_simulation.gif"), fps = 5)
30 println("Анимация сохранена в plots/philosophers_simulation.gif")
31
```

Figure 4: Скрипт `dining_philosophers_animation.jl`: построение анимации маркировки для $N = 3$

сети из датафрейма выбираются столбцы Eat_1-Eat_5, строятся графики состояний «ест» по времени для классической сети и сети с арбитром, оба графика объединяются в макет 2×1 размером 800×600 и сохраняются в final_report.png ([рис. @fig-005]).



```

1  using DrWatson
2  @quickactivate "project"
3  using DataFrames, CSV, Plots
4
5  df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
6  df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
7  N = 5
8
9  eat_cols = [Symbol("Eat_$i") for i = 1:N]
10
11 p1 = plot(
12     df_classic.time,
13     Matrix(df_classic[:, eat_cols]),
14     label = ["Φ $i" for i = 1:N],
15     xlabel = "Время",
16     ylabel = "Ест (1/0)",
17     title = "Классическая сеть",
18 )
19 p2 = plot(
20     df_arbiter.time,
21     Matrix(df_arbiter[:, eat_cols]),
22     label = ["Φ $i" for i = 1:N],
23     xlabel = "Время",
24     ylabel = "Ест (1/0)",
25     title = "Сеть с арбитром",
26 )
27 p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
28 savefig(plotsdir("final_report.png"))
29
30 println("Отчёт сохранён в plots/final_report.png")
31

```

Figure 5: Скрипт dining_philosophers_report.jl: построение сравнительного отчёта

Скрипт параметрического исследования

В скрипте dining_philosophers_parametrized_literate.jl задан набор параметров param_sets: (N = 3, tmax = 30.0), (N = 5, tmax = 50.0), (N = 7, tmax = 70.0). Функция run_experiment(N, tmax) для каждого набора строит классическую сеть и сеть с арбитром, выполняет моделирование, сохраняет CSV-файлы dining_classic_N\$(N).csv и dining_arbiter_N\$(N).csv, проверяет наличие тупика для обоих вариантов и возвращает результат в виде именованного кортежа ([рис. @fig-006]).

```

1 using DrWatson
2 @quickactivate "project"
3 include(scrdir("DiningPhilosophers.jl"))
4 using DiningPhilosophers
5 using DataFrames, CSV, Plots
6
7 param_sets = [
8     (N = 3, tmax = 30.0),
9     (N = 5, tmax = 50.0),
10    (N = 7, tmax = 70.0),
11 ]
12
13 function run_experiment(N, tmax)
14     println("N = $N, tmax = $tmax")
15
16     net_classic, u0_classic, _ = build_classical_network(N)
17     df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
18     CSV.write(datadir("dining_classic_N$(N).csv"), df_classic)
19     dead_classic = detect_deadlock(df_classic, net_classic)
20
21     net_arb, u0_arb, _ = build_arbiter_network(N)
22     df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
23     CSV.write(datadir("dining_arbiter_N$(N).csv"), df_arb)
24     dead_arb = detect_deadlock(df_arb, net_arb)
25
26     return (N = N, tmax = tmax, deadlock_classic = dead_classic, deadlock_arbiter = dead_arb)
27 end
28
29 results = []
30 for params in param_sets
31     result = run_experiment(params.N, params.tmax)
32     push!(results, result)
33 end
34

```

Figure 6: Скрипт параметрического исследования: набор параметров и функция `run_experiment`

Установленные пакеты

Проверка состояния проекта командой `Pkg.status()` показала, что в `Project.toml` установлены пакеты: `CSV v0.10.16`, `DataFrames v1.8.2`, `DrWatson v2.19.1`, `OrdinaryDiffEq v7.0.0`, `Plots v1.41.6` ([рис. @fig-007]).

```

julia> Pkg.status()
Status ~/2026-1--study--simulation-modeling/Labs/lab05/project/Project.toml
 [336ed68f] CSV v0.10.16
 [a93c6f00] DataFrames v1.8.2
 [634d3b9d] DrWatson v2.19.1
 [1dea7af3] OrdinaryDiffEq v7.0.0
 [91a5bcd0] Plots v1.41.6

```

Figure 7: Состояние установленных пакетов проекта lab05

Базовый эксперимент: классическая сеть и сеть с арбитром

Выполнен `include("scripts/dining_philosophers.jl")`. Для классической сети (без арбитра) обнаружен тупик: `Deadlock обнаружен: true`. Для сети с арбитром тупик не обнаружен: `Deadlock обнаружен: false`. При построении графика для сети с арбитром появилось предупреждение `Could not create decoration from factory! Running with no decorations`, не влияющее на результат. Итоговый график сети с арбитром сохранён в `arbiter_simulation.png` ([рис. @fig-008]).


```

julia> include("scripts/dining_philosophers.jl")
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
"/home/dgavdadaev/2026-1-study-simulation-modeling/labs/lab05/project/plots/arbiter_simulation.png"

```

Figure 8: Результат базового эксперимента: deadlock в классической сети и его отсутствие в сети с арбитром

В каталоге data появились подкаталоги exp_pro, exp_raw, sims, а также файлы результатов dining_arbiter.csv и dining_classic.csv (рис. @fig-009).

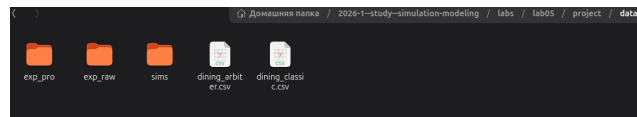


Figure 9: Файлы данных проекта после базового эксперимента

Анализ графика классической сети (рис. @fig-010)

На графике classic_simulation.png (рис. @fig-010) видно четыре панели — Think, Hungry, Eat и Fork states. Моделирование останавливается около $t \approx 2.6$ из-за тупика. На панели Eat states видно, что философ Eat_3 ест дважды короткими интервалами (примерно $t \approx 0.75-1.0$ и $t \approx 1.2-2.5$), а остальные четыре философа (Eat_1, Eat_2, Eat_4, Eat_5) за всё время моделирования вообще не приступают к еде — их кривые остаются на нуле. На панели Fork states видно, что к моменту тупика (после $t \approx 1.0$) большинство вилок (Fork_1-Fork_4) фиксируются в положении 0 (захвачены и не освобождаются), а свободной остаётся только одна вилка (Fork_5, убывающая до нуля к $t \approx 2.5$), что соответствует классической схеме deadlock: каждый философ держит одну вилку и ждёт вторую.

Анализ графика сети с арбитром (рис. @fig-011)

На графике arbiter_simulation.png (рис. @fig-011) видно, что моделирование продолжается весь интервал $t \in [0, 50]$ без остановки. На панели Eat states все пять философов (Eat_1-Eat_5) регулярно переходят в состояние «ест» в виде острых треугольных импульсов, равномерно распределённых по всему времени моделирования, без длительных периодов простоя. На панели Fork states вилки (Fork_1-Fork_5) также активно

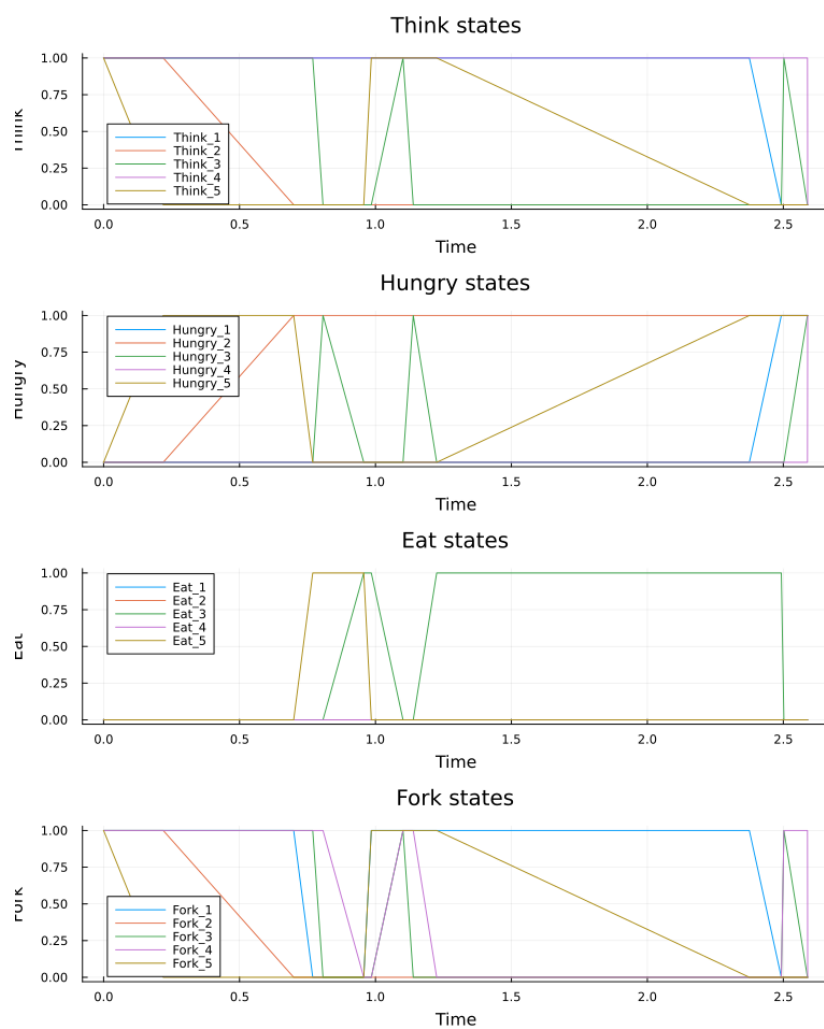


Figure 10: classic_simulation.png: динамика маркировки классической сети для $N = 5$

переключаются между 0 и 1 на протяжении всего интервала, что говорит об отсутствии захваченных навсегда ресурсов. Это подтверждает результат `Deadlock обнаружен: false` — арбитр устраняет тупиковую ситуацию и обеспечивает непрерывную работу системы.

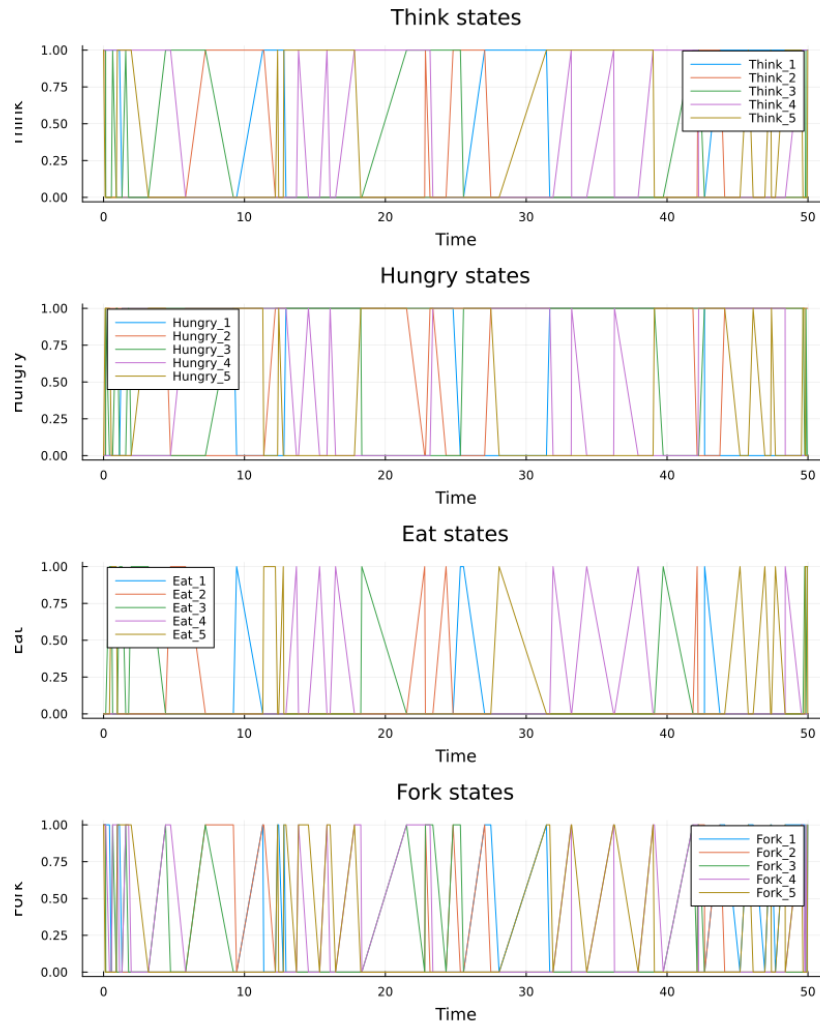
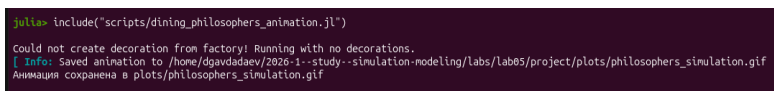


Figure 11: `arbiter_simulation.png`: динамика маркировки сети с арбитром для $N = 5$

Построение анимации маркировки

Выполнен `include("scripts/dining_philosophers_animation.jl")`. В терминале появилось сообщение `Could not create decoration from factory! Running with no decorations`, после чего анимация успешно сохранена: [Info: Saved animation to ...plots/philosophers_simulation.gif, Анимация сохранена в plots/philosophers_simulation.gif ([рис. @fig-012])).



```
julia> include("scripts/dining_philosophers_animation.jl")
Could not create decoration from factory! Running with no decorations.
[ Info: Saved animation to /home/dgavdadaev/2026-1--study--simulation-modeling/labs/lab05/project/plots/philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
```

Figure 12: Результат генерации анимации philosophers_simulation.gif

В каталоге plots появились файлы `arbiter_simulation.png`, `classic_simulation.png` и `philosophers_simulation.gif` ([рис. @fig-013]).

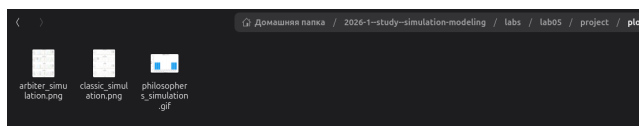


Figure 13: Файлы графиков и анимации в каталоге plots

На кадре анимации для момента «Время = 0.0» видно начальное состояние сети для $N = 3$: позиции `Think_1`, `Think_2`, `Think_3` и `Fork_1`, `Fork_2`, `Fork_3` содержат по одной фишке (значение 1), а позиции `Hungry_1`-`Hungry_3` и `Eat_1`-`Eat_3` пусты (значение 0) — все философы находятся в состоянии «думает», все вилки свободны ([рис. @fig-014])).

Формирование итогового отчёта

Выполнен `include("scripts/dining_philosophers_report.jl")`. В терминале появилось сообщение `Отчёт сохранён в plots/final_report.png` ([рис. @fig-015])).

После выполнения в каталоге plots появился файл `final_report.png` рядом с ранее созданными `arbiter_simulation.png`, `classic_simulation.png` и `philosophers_simulation.gif` ([рис. @fig-016])).

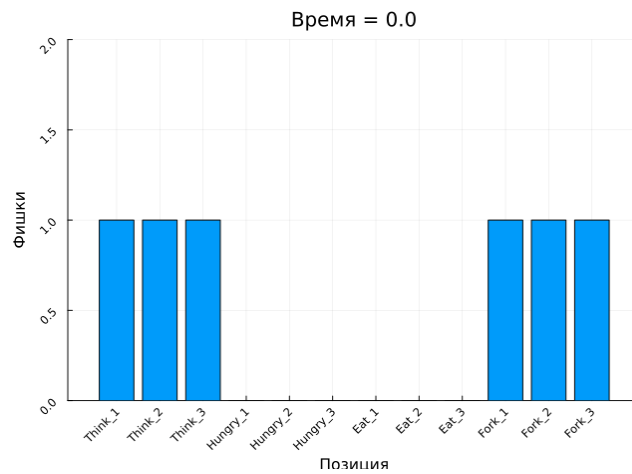


Figure 14: philosophers_simulation.gif: начальный кадр анимации маркировки сети для $N = 3$, момент времени 0.0

```
julia> include("scripts/dining_philosophers_report.jl")
Отчёт сохранён в plots/final_report.png
Could not create decoration from factory! Running with no decorations
```

Figure 15: Результат выполнения скрипта формирования итогового отчёта

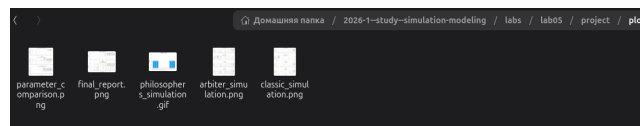


Figure 16: Каталог plots с файлом final_report.png

Анализ итогового сравнительного графика ([рис. @fig-017])

На графике `final_report.png` ([рис. @fig-017]) верхняя панель «Классическая сеть» показывает состояния «ест» (`Eat_1-Eat_5`) для $N = 5$ на интервале $t \in [0, 2.5]$. Видно, что только два философа успевают поест: один — короткий импульс длительностью около 0.2 в районе $t \approx 0.3-0.5$, другой — более длинный импульс длительностью около 1.0 в районе $t \approx 0.4-1.4$, после чего все кривые опускаются в 0 и остаются на этом уровне до конца интервала — система входит в тупик, остальные три философа не успевают поесть ни разу. Нижняя панель «Сеть с арбитром» на интервале $t \in [0, 50]$ показывает множество регулярных треугольных импульсов состояния «ест» для всех пяти философов, равномерно распределённых по всему времени без длительных провалов — арбитр обеспечивает поочерёдный доступ всех философов к ресурсам без блокировок.

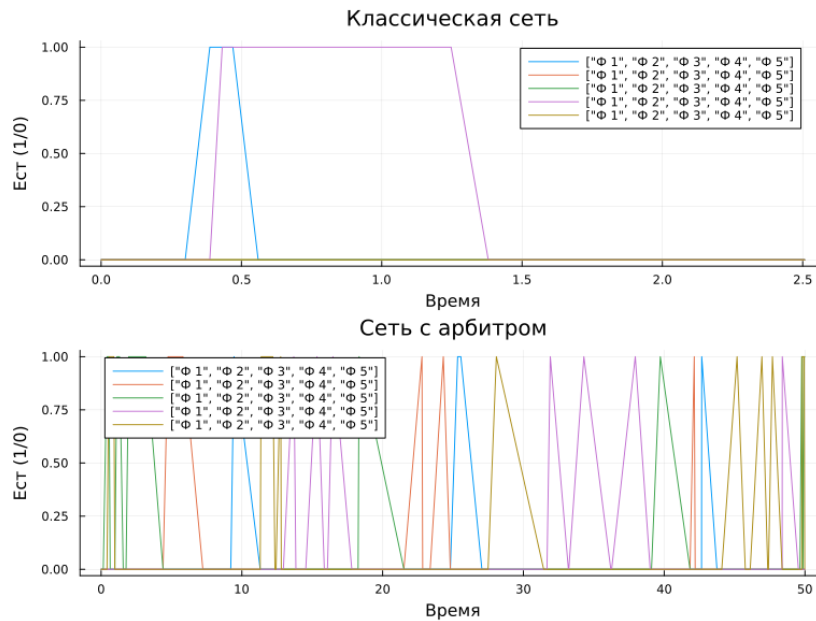
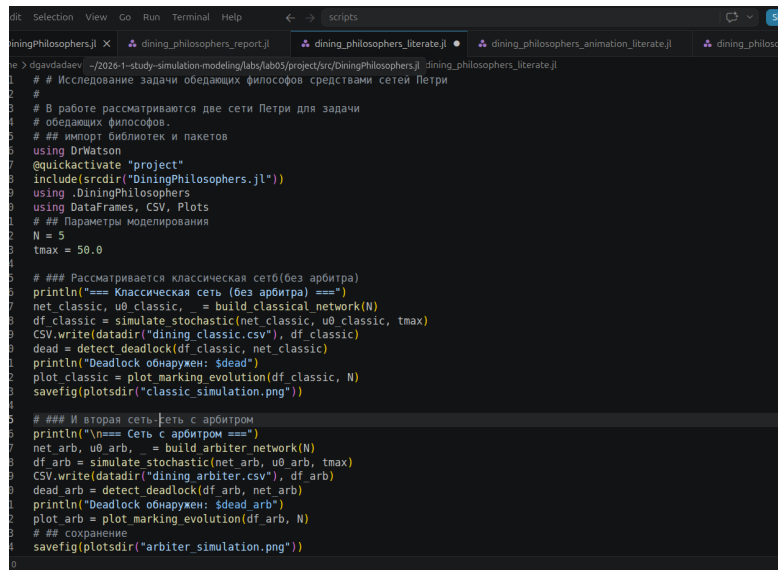


Figure 17: `final_report.png`: сравнение динамики состояний «ест» для классической сети и сети с арбитром, $N = 5$

Литературное программирование

Все скрипты были преобразованы в литературный стиль. Скрипт `dining_philosophers_literate.jl` содержит заголовок

«Исследование задачи обедающих философов средствами сетей Петри» и комментарий «В работе рассматриваются две сети Петри для задачи обедающих философов», разделы «Параметры моделирования» ($N = 5$, $t_{\max} = 50.0$) и «Рассматривается классическая сеть (без арбитра)» ([рис. @fig-018]).



```

1 # Исследование задачи обедающих философов средствами сетей Петри
2 #
3 # В работе рассматриваются две сети Петри для задачи
4 # обедающих философов.
5 ## импорт библиотек и пакетов
6 using DrWatson
7 @quickactivate "project"
8 include(scrdir("DiningPhilosophers.jl"))
9 using .DiningPhilosophers
10 using DataFrames, CSV, Plots
11 ## Параметры моделирования
12 N = 5
13 tmax = 50.0
14
15 ### Рассматривается классическая сеть (без арбитра)
16 println("== Классическая сеть (без арбитра) ==")
17 net_classic, u0_classic, _ = build_classical_network(N)
18 df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
19 CSV.write(datadir("dining_classic.csv"), df_classic)
20 dead = detect_deadlock(df_classic, net_classic)
21 println("Deadlock обнаружен: $dead")
22 plot_classic = plot_marking_evolution(df_classic, N)
23 savefig(plotsdir("classic_simulation.png"))
24
25 ## И вторая сеть: сеть с арбитром
26 println("\n== Сеть с арбитром ==")
27 net_arb, u0_arb, _ = build_arbiter_network(N)
28 df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
29 CSV.write(datadir("dining_arbiter.csv"), df_arb)
30 dead_arb = detect_deadlock(df_arb, net_arb)
31 println("Deadlock обнаружен: $dead_arb")
32 plot_arb = plot_marking_evolution(df_arb, N)
33 ## сохранение
34 savefig(plotsdir("arbiter_simulation.png"))

```

Figure 18: dining_philosophers_literate.jl: литературная версия базового скрипта

Скрипт dining_philosophers_animation_literate.jl содержит заголовок «Анимация изменения маркировки сети Петри» и подзаголовок «Визуализация динамики работы модели обедающих философов», разделы «Параметры моделирования» ($N = 3$, $t_{\max} = 30.0$) и «Выполнение моделирования» ([рис. @fig-019]).

Скрипт dining_philosophers_parametrized_literate.jl содержит заголовок «Параметрическое исследование сети Петри» и подзаголовок «Сравнение поведения модели при различном количестве философов», раздел «Набор параметров» с тем же набором param_sets, что и в базовом скрипте параметрического исследования ([рис. @fig-020]).

С помощью команд Literate.notebook(...) для всех трёх литературных скриптов (dining_philosophers_literate.jl, dining_philosophers_animation_literate.jl, dining_philosophers_parametrized_literate.jl), а также для скрипта dining_philosophers_report_literate.jl, сгенерированы Jupyter Notebook-файлы в каталоге notebooks

```

1  > dgaudadev > 2026-1-study-simulation-modeling > labs > lab05 > project > scripts > dining_philosophers_animation_iterate.jl
2  # # Анимация изменения маркировки сети Петри
3  #
4  # Визуализация динамики работы модели обедающих философов
5
6  using DrWatson
7  @quickactivate "project"
8  include(scrdir("DiningPhilosophers.jl"))
9  using .DiningPhilosophers
10 using Plots, Random
11
12 ## Параметры моделирования
13 N = 3
14 tmax = 30.0
15
16 ## Выполнение моделирования
17 net, u0, names = build_classical_network(N)
18
19 Random.seed!(123)
20 df = simulate_stochastic(net, u0, tmax)
21
22 ## Построение анимации
23 anim = @animate for row in eachrow(df)
24     u = [row[col] for col in propertynames(row) if col != :time]
25     bar(
26         1:length(u),
27         u,
28         legend = false,
29         ylims = (0, maximum(u0) + 1),
30         xlabel = "Позиция",
31         ylabel = "Философы",
32         title = "Время = $(round(row.time, digits=2))",
33     )
34     xticks!(1:length(u), string.(names), rotation = 45)
35 end

```

Figure 19: dining_philosophers_animation_iterate.jl: литературная версия скрипта анимации

```

1  home > dgaudadev > 2026-1-study-simulation-modeling > labs > lab05 > project > scripts > dining_philosophers_parametrized_iterate.jl
2  # # Параметрическое исследование сети Петри
3  #
4  # Сравнение поведения модели при различном количестве философов
5
6  using DrWatson
7  @quickactivate "project"
8  include(scrdir("DiningPhilosophers.jl"))
9  using .DiningPhilosophers
10 using DataFrames, CSV, Plots
11
12 ## Набор параметров
13
14 param_sets = [
15     (N = 3, tmax = 30.0),
16     (N = 5, tmax = 50.0),
17     (N = 7, tmax = 70.0),
18 ]
19
20 ## Проведение эксперимента
21
22 function run_experiment(N, tmax)
23     println("N = $N, tmax = $tmax")
24
25     net_classic, u0_classic, = build_classical_network(N)
26     df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
27     CSV.write(datadir("dining_classic_N$(N).csv"), df_classic)
28     dead_classic = detect_deadlock(df_classic, net_classic)
29
30     net_arb, u0_arb, = build_arbiter_network(N)
31     df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
32     CSV.write(datadir("dining_arbiter_N$(N).csv"), df_arb)
33     dead_arb = detect_deadlock(df_arb, net_arb)
34
35     return (N = N, tmax = tmax, deadlock_classic = dead_classic, deadlock_arbiter = dead_arb)
36 end

```

Figure 20: dining_philosophers_parametrized_iterate.jl: литературная версия скрипта параметрического исследования

([рис. @fig-021]).

```
Julia> Literate.notebook("scripts/dining_philosophers_animation_iterate.jl", "notebooks")
[ Info: generating notebook from "/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_animation_iterate.jl"
[ Info: executing notebook "dining_philosophers_animation_iterate.ipynb"
[ Info: writing result to "/2026-1-study-simulation-modeling/labs/lab05/project/notebooks/dining_philosophers_animation_iterate.ipynb"
/home/dgavdadaev/2026-1-study-simulation-modeling/labs/lab05/project/notebooks/dining_philosophers_animation_iterate.ipynb

Julia> Literate.notebook("scripts/dining_philosophers_parametrized_iterate.jl", "notebooks")
[ Info: generating notebook from "/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_parametrized_iterate.jl"
[ Info: executing notebook "dining_philosophers_parametrized_iterate.ipynb"
[ Info: writing result to "/2026-1-study-simulation-modeling/labs/lab05/project/notebooks/dining_philosophers_parametrized_iterate.ipynb"
/home/dgavdadaev/2026-1-study-simulation-modeling/labs/lab05/project/notebooks/dining_philosophers_parametrized_iterate.ipynb

Julia> Literate.notebook("scripts/dining_philosophers_report_iterate.jl", "notebooks")
[ Info: generating notebook from "/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_report_iterate.jl"
[ Info: executing notebook "dining_philosophers_report_iterate.ipynb"
[ Info: writing result to "/2026-1-study-simulation-modeling/labs/lab05/project/notebooks/dining_philosophers_report_iterate.ipynb"
/home/dgavdadaev/2026-1-study-simulation-modeling/labs/lab05/project/notebooks/dining_philosophers_report_iterate.ipynb
```

Figure 21: Генерация Jupyter Notebook-файлов с помощью `Literate.notebook`

Аналогично, с помощью `Literate.markdown(..., "markdown"; flavor = Literate.QuartoFlavor())` для всех четырёх скриптов сгенерированы Quarto-документы (.qmd) в каталоге `markdown` ([рис. @fig-022]).

```
Julia> Literate.markdown("scripts/dining_philosophers_iterate.jl", "markdown"; flavor = Literate.QuartoFlavor())
[ Info: generating markdown page from "/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_iterate.jl"
[ Info: writing result to "/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_iterate.qmd"
/home/dgavdadaev/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_iterate.qmd

Julia> Literate.markdown("scripts/dining_philosophers_animation_iterate.jl", "markdown"; flavor = Literate.QuartoFlavor())
[ Info: generating markdown page from "/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_animation_iterate.jl"
[ Info: writing result to "/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_animation_iterate.qmd"
/home/dgavdadaev/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_animation_iterate.qmd

Julia> Literate.markdown("scripts/dining_philosophers_parametrized_iterate.jl", "markdown"; flavor = Literate.QuartoFlavor())
[ Info: generating markdown page from "/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_parametrized_iterate.jl"
[ Info: writing result to "/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_parametrized_iterate.qmd"
/home/dgavdadaev/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_parametrized_iterate.qmd

Julia> Literate.markdown("scripts/dining_philosophers_report_iterate.jl", "markdown"; flavor = Literate.QuartoFlavor())
[ Info: generating markdown page from "/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_report_iterate.jl"
[ Info: writing result to "/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_report_iterate.qmd"
/home/dgavdadaev/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_report_iterate.qmd
```

Figure 22: Генерация Quarto-документов с помощью `Literate.markdown` с флагом `QuartoFlavor`

Jupyter Notebook: анимация

В сгенерированном Jupyter Notebook `dining_philosophers_animation_iterate.ipynb` после выполнения ячеек видна таблица результата моделирования: датафрейм размером 4×13 со столбцами `time`, `Think_1-Think_3`, `Hungry_1-Hungry_3`, `Eat_1-Eat_3`, `Fork_1-Fork_3` для $N = 3$. В первой строке (`time = 0.0`) все `Think_i = 1.0` и `Fork_i = 1.0`, остальные столбцы равны `0.0`, что соответствует начальному состоянию сети ([рис. @fig-023]).

Jupyter Notebook: базовый эксперимент

В Jupyter Notebook `dining_philosophers_iterate.ipynb` отображается заголовок «Исследование задачи обедающих философов средствами сетей Петри» с подзаголовком «В работе рассматриваются

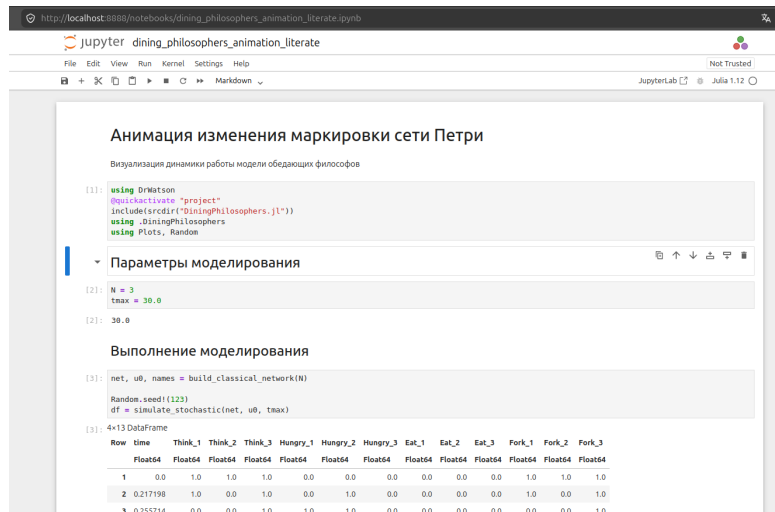


Figure 23: Jupyter Notebook dining_philosophers_animation_iterate.ipynb: датафрейм результатов для N = 3

две сети Петри для задачи обедающих философов», раздел «Параметры моделирования» с N = 5, tmax = 50.0 и раздел «Рассматривается классическая сеть (без арбитра)» с выводом сообщения о наличии тупика ([рис. @fig-024]).

Jupyter Notebook: параметрическое исследование

В Jupyter Notebook dining_philosophers_parametrized_iterate.ipynb отображается заголовок «Параметрическое исследование сети Петри» с подзаголовком «Сравнение поведения модели при различном количестве философов», раздел «Набор параметров» с выводом 3-element Vector{@NamedTuple{N::Int64, tmax::Float64}}: (N = 3, tmax = 30.0), (N = 5, tmax = 50.0), (N = 7, tmax = 70.0) и раздел «Проведение эксперимента» с функцией run_experiment ([рис. @fig-025]).

Jupyter Notebook: формирование отчёта

В Jupyter Notebook dining_philosophers_report_iterate.ipynb отображается заголовок «Формирование итогового отчёта» с подзаголовком «Построение сравнительных графиков для двух вариантов сети петри», раздел «Загрузка результатов моделирования» с выводом 5-element Vector{Symbol}: :Eat_1, :Eat_2, :Eat_3, :Eat_4, :Eat_5 и раздел «Построение графиков» ([рис. @fig-026]).

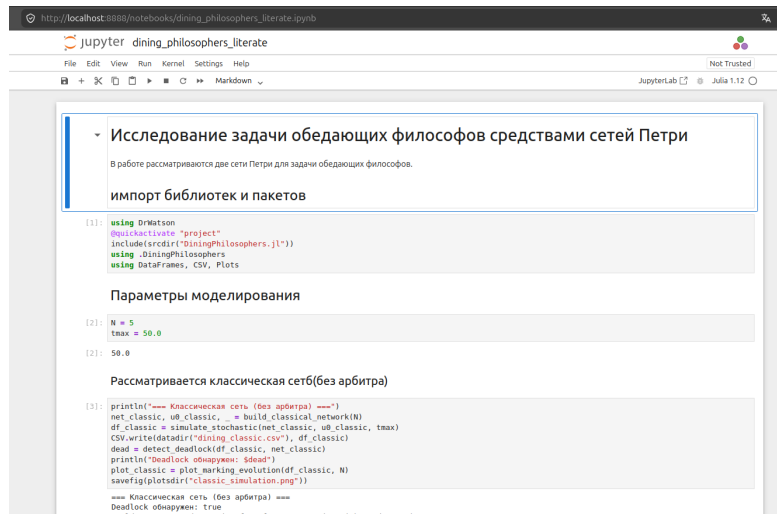


Figure 24: Jupyter Notebook dining_philosophers_literate.ipynb: заголовок и базовые параметры

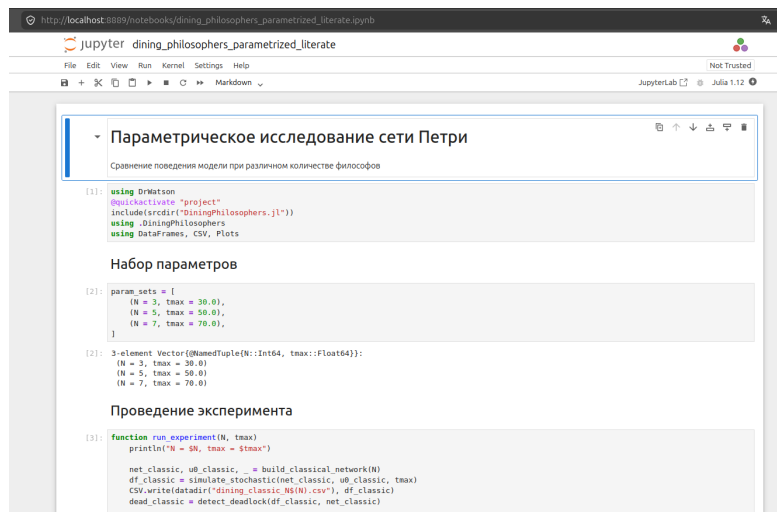


Figure 25: Jupyter Notebook dining_philosophers_parametrized_literate.ipynb: заголовок, набор параметров и функция run_experiment

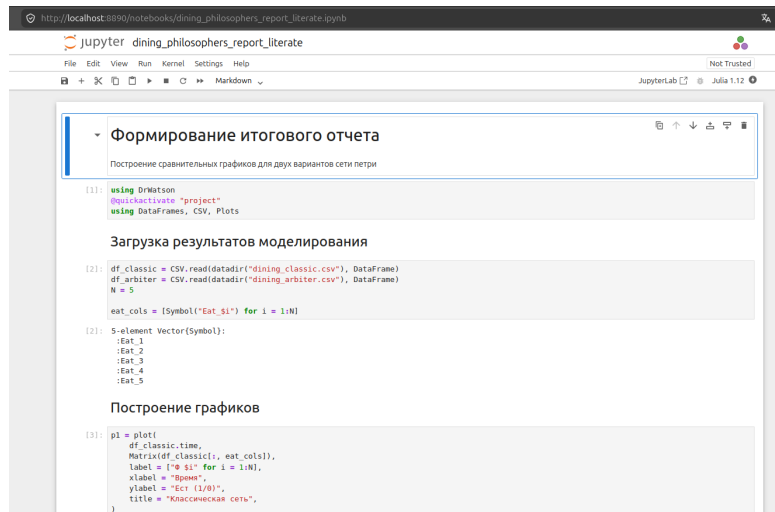


Figure 26: Jupyter Notebook dining_philosophers_report_literate.ipynb: заголовок, загрузка результатов и построение графиков

Quarto-документы (HTML)

Сгенерированный Quarto HTML-документ dining_philosophers_animation_literate.html отображает заголовок «Анимация изменения маркировки сети Петри», подзаголовок «Визуализация динамики работы модели обедающих философов» и таблицу датафрейма 4×13 для $N = 3$ с тем же содержимым, что и в Jupyter Notebook ([рис. @fig-027]).

Документ dining_philosophers_parametrized_literate.html отображает заголовок «Параметрическое исследование сети Петри», подзаголовок «Сравнение поведения модели при различном количестве философов», набор параметров param_sets и начало функции run_experiment ([рис. @fig-028]).

Документ dining_philosophers_report_literate.html отображает заголовок «Формирование итогового отчёта», подзаголовок «Построение сравнительных графиков для двух вариантов сети петри», загрузку CSV-файлов dining_classic.csv и dining_arbiter.csv, список столбцов Eat_1-Eat_5 и начало построения графика p1 для классической сети ([рис. @fig-029]).

Документ dining_philosophers_literate.html отображает заголовок «Исследование задачи обедающих философов средствами сетей Петри», подзаголовок «В работе рассматриваются две сети Петри для задачи обедающих философов», раздел «импорт библиотек и пакетов», раздел «Параметры моделирования» с $N = 5$, $t_{\max} = 50.0$ и раздел «Рассматривается классическая

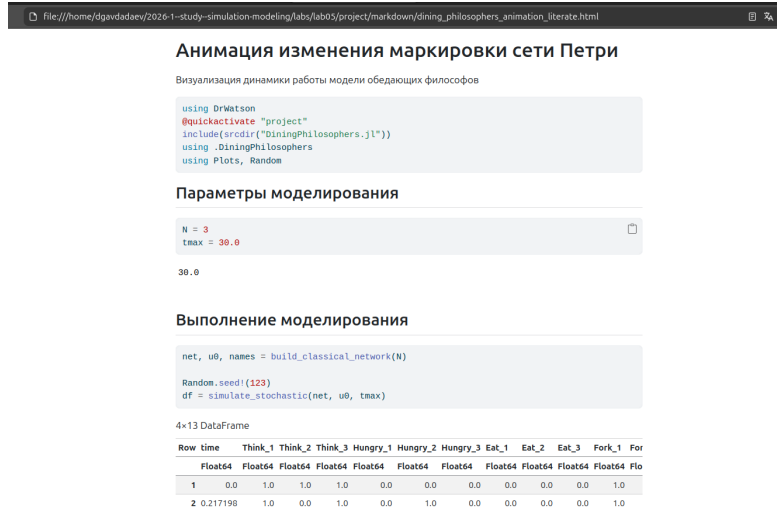


Figure 27: dining_philosophers_animation_iterate.html: HTML-версия Quarto-документа с анимацией

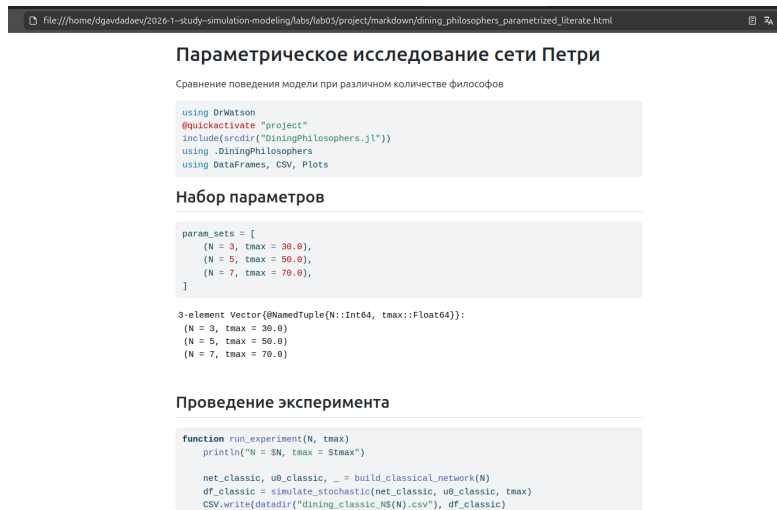


Figure 28: dining_philosophers_parametrized_iterate.html: HTML-версия Quarto-документа с параметрическим исследованием

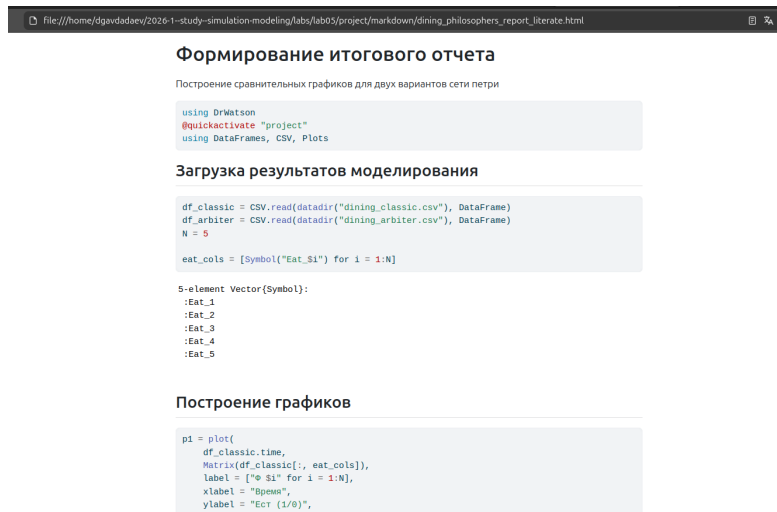


Figure 29: dining_philosophers_report_literate.html: HTML-версия Quarto-документа с итоговым отчётом

сеть (без арбитра)» с кодом построения сети, моделирования, сохранения CSV и проверки тупика ([рис. @fig-030]).

Параметрическое исследование

Выполнен `include("scripts/dining_philosophers_parametrized_literate.jl")`. Для каждого набора параметров в терминал выводится строка `N = $N, tmax = $tmax: N = 3, tmax = 30.0, N = 5, tmax = 50.0, N = 7, tmax = 70.0`. Результат эксперимента — датафрейм 3×4 со столбцами `N`, `tmax`, `deadlock_classic`, `deadlock_arbiter`. Для всех трёх значений `N` (3, 5, 7) классическая сеть приходит к тупику (`deadlock_classic = true`), а сеть с арбитром во всех случаях работает без тупика (`deadlock_arbiter = false`). При выполнении возникли два предупреждения о неоднозначности присвоения переменных `N` и `eat_cols` в мягкой области видимости (soft scope), не повлиявшие на результат вычислений ([рис. @fig-031]).

Анализ графика параметрического сравнения ([рис. @fig-032])

На графике `parameter_comparison.png` ([рис. @fig-032]) представлены три панели для классической сети при `N = 3`, `N = 5` и `N = 7`. При `N = 3` моделирование останавливается уже к `t ≈ 0.7`: единственный треугольный импульс приёма пищи (Φ

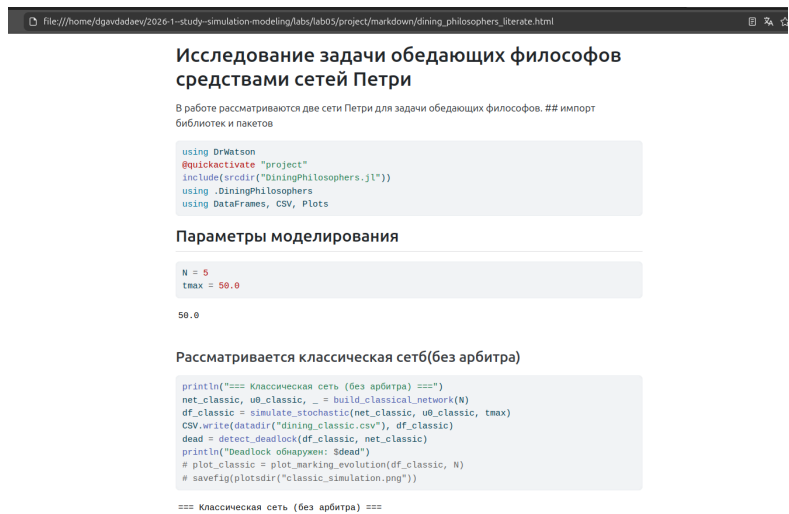


Figure 30: dining_philosophers_literate.html: HTML-версия Quarto-документа с базовым экспериментом



Figure 31: Результаты параметрического исследования: таблица deadlock_classic / deadlock_arbiter для N = 3, 5, 7

3) длительностью около 0.2 — после чего наступает тупик. При $N = 5$ интервал моделирования увеличивается до $t \approx 17.5$, видно пять-шесть коротких импульсов состояния «ест» в начале интервала ($t < 5$) и ещё два более длинных импульса (длительностью около 1.5–2.0) около $t \approx 6.5$ –8 и $t \approx 14$ –16.5, после чего система останавливается. При $N = 7$ интервал самый короткий — моделирование останавливается уже к $t \approx 6.5$, видны лишь два импульса приёма пищи: длительностью около 1.2 в районе $t \approx 0.6$ –1.8 и длительностью около 3.0 в районе $t \approx 2.3$ –5.3, после чего сразу наступает тупик. Таким образом увеличение числа философов N в классической сети не увеличивает время до тупика монотонно — для $N = 7$ тупик наступает заметно быстрее, чем для $N = 5$, что объясняется возросшей конкуренцией за общий пул вилок.

Выводы

Реализована модель сетей Петри для задачи обедающих философов в двух вариантах — классическом и с арбитром, на базе структуры PetriNet с матрицей инцидентности. Базовый эксперимент при $N = 5$ показал, что классическая сеть приходит к тупику примерно к $t \approx 1.4$ –2.6 ([рис. @fig-010], [рис. @fig-017]): из пяти философов поест успевают только два, после чего четыре из пяти вилок (Fork_1–Fork_4) фиксируются захваченными. Сеть с арбитром на том же интервале $t \in [0, 50]$ работает без тупика — все пять философов регулярно принимают пищу на протяжении всего времени моделирования ([рис. @fig-011], [рис. @fig-017]). Построена анимация изменения маркировки сети для $N = 3$, сохранённая в philosophers_simulation.gif ([рис. @fig-014]). Параметрическое исследование при $N = 3, 5, 7$ подтвердило, что классическая сеть приводит к тупику во всех трёх случаях (deadlock_classic = true), причём время до тупика составило $t \approx 0.7$ для $N = 3$, $t \approx 17.5$ для $N = 5$ и $t \approx 6.5$ для $N = 7$ ([рис. @fig-031], [рис. @fig-032]), а сеть с арбитром во всех случаях работала без тупика (deadlock_arbiter = false). Все скрипты преобразованы в литературный стиль с помощью Literate.jl, сгенерированы Jupyter Notebook и Quarto-документы (HTML) для базового эксперимента, анимации, параметрического исследования и формирования итогового отчёта.

```
{ {< include ../project/markdown/dining_philosophers_literate/dining_philosophers_literate.qm
>}}
```

```
{ {< include ../project/markdown/dining_philosophers_animation_literate/dining_philosophers_a
>}}
```

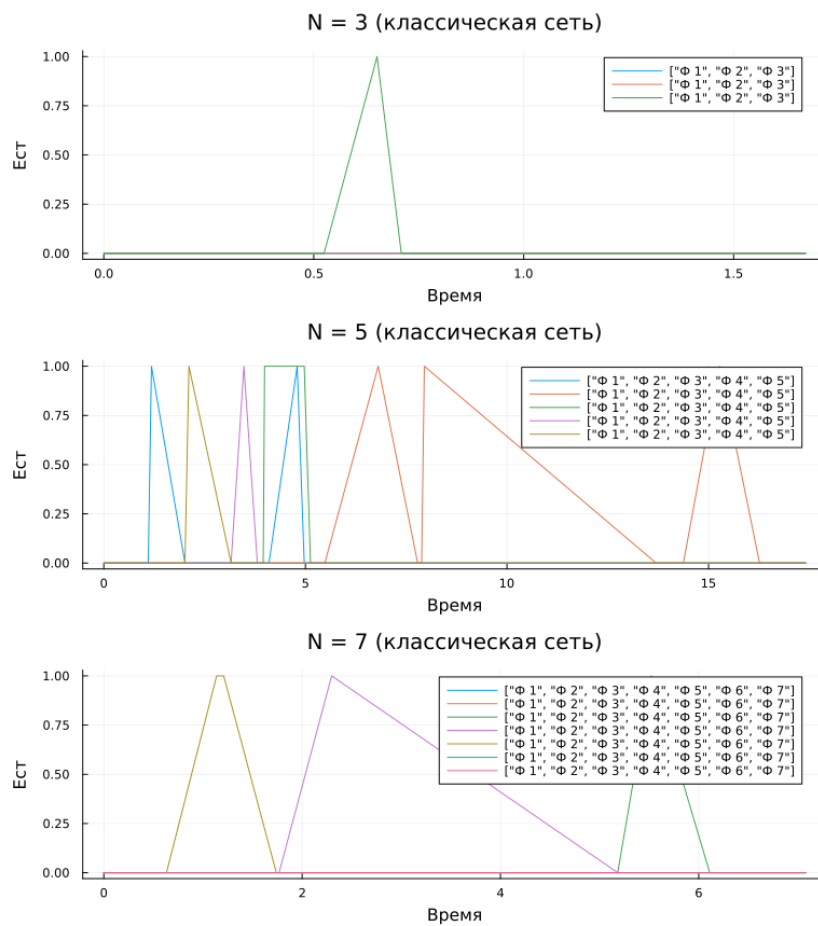



Figure 32: parameter_comparison.png: сравнение динамики состояний «ест» в классической сети при $N = 3, 5, 7$

```
{{< include ../project/markdown/dining_philosophers_parametrized_literate/dining_philosophers  
>}}  
{{< include ../project/markdown/dining_philosophers_report_literate/dining_philosophers_repo  
>}}
```

Список литературы

1. Кулябов Д. С., Королькова А. В. Имитационное моделирование. Практикум. — М.: РУДН, 2024.
2. Dijkstra E. W. Hierarchical ordering of sequential processes // Acta Informatica. — 1971.
3. Peterson J. L. Petri net theory and the modeling of systems. — Prentice Hall, 1981.
4. Murata T. Petri nets: Properties, analysis and applications // Proceedings of the IEEE. — 1989.
5. Datseris G., Vahdati A. R., DuBois T. C. Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity // SIMULATION. — 2022.